Звіт до лабораторної роботи: ієрархія класів, успадкування та поліморфізм

Тема: Успадкування, віртуальні методи, `override` та `new` (C#) **Варіант:** 7
Виконав: Зінчук Д. Ю.

1. Посилання на репозиторій

GitHub: https://github.com/zinchukdiipz25-wq/OOP---Zinchuk-labs

Проєкт створено командою:

```
dotnet new console -o OOP-Zinchuk/lab4v7
```

2. Постановка завдання

Реалізувати ієрархію класів `Person` → `Student`, `Professor` згідно з варіантом 7:

- **Person (базовий):** приватні поля, публічні властивості `Name`, `Age`, конструктор, віртуальний метод `Greet()`, звичайний (невіртуальний) метод `GetRole()`;
- **Student (похідний):** властивість `StudentId`, `override Greet()`, метод `Study()`, конструктор із викликом `base(...)`, метод `new GetRole()`;
- **Professor (похідний):** властивість `Department`, `override Greet()`, метод `Teach()`, конструктор із викликом `base(...)`.

У методі `Main` потрібно створити об'єкти базового та похідних класів, продемонструвати поліморфізм, різницю між `override` та `new` і вивести результати в консоль.

3. Код програми

```
using System;

class Person
{
    // Приватні поля
    private string _name;
    private int _age;

    // Публічні властивості з валідацією
```

```
    public string Name
    {
        get { return _name; }
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Ім'я не може бути порожнім!");

            _name = value;
        }
    }

    public int Age
    {
        get { return _age; }
        set
        {
            if (value < 0 || value > 120)
                throw new ArgumentOutOfRangeException(
                    nameof(value), "Вік має бути від 0 до 120!");

            _age = value;
        }
    }

    // Конструктор
    public Person(string name, int age)
    {
        Name = name;
        Age = age;
    }

    // Віртуальний метод: може бути перевизначений у похідних класах
    public virtual void Greet()
    {
        Console.WriteLine($"Привіт, мене звати {Name}. Мій вік: {Age}.");
    }

    // Звичайний метод (не віртуальний)
    public string GetRole()
    {
        return "Особа";
    }
}

class Student : Person
```

```
{
    private string _studentId;

    public string StudentId
    {
        get { return _studentId; }
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Номер студентського не може бути
порожнім!");

            _studentId = value;
        }
    }

    // Виклик конструктора базового класу
    public Student(string name, int age, string studentId) : base(name, age)
    {
        StudentId = studentId;
    }

    // Перевизначення віртуального методу
    public override void Greet()
    {
        Console.WriteLine($"Привіт, я студент {Name}, студентський квиток
{StudentId}.");
    }

    // Власний унікальний метод
    public void Study()
    {
        Console.WriteLine($"{Name} вивчає об'єктно-орієнтоване
програмування.");
    }

    // Приховування методу базового класу (не override!)
    public new string GetRole()
    {
        return "Студент";
    }
}

class Professor : Person
{
    private string _department;
```

```
    public string Department
    {
        get { return _department; }
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Кафедра не може бути
порожньою!");

            _department = value;
        }
    }

    // Виклик конструктора базового класу
    public Professor(string name, int age, string department) : base(name,
age)
    {
        Department = department;
    }

    // Перевизначення віртуального методу
    public override void Greet()
    {
        Console.WriteLine($"Доброго дня, я професор {Name}, кафедра:
{Department}.");
    }

    // Власний унікальний метод
    public void Teach()
    {
        Console.WriteLine($"{Name} читає лекцію на кафедрі: {Department}.");
    }
}

class Program
{
    static void Main()
    {
        Console.OutputEncoding = System.Text.Encoding.UTF8;

        Person person = new Person("Олег", 40);
        Student student = new Student("Марія", 19, "ST-2025-07");
        Professor professor = new Professor("Іван Петренко", 52,
"Інформаційні технології");
```

```
        Console.WriteLine("=== Поліморфізм (override) ===");
        Person[] people = { person, student, professor };
        foreach (Person p in people)
        {
            p.Greet();
        }

        Console.WriteLine();

        Console.WriteLine("=== Власні методи похідних класів ===");
        student.Study();
        professor.Teach();

        Console.WriteLine();

        Console.WriteLine("=== Різниця між override та new ===");
        Person studentAsPerson = student;

        Console.WriteLine("Greet() через посилання типу Person (override):");
        studentAsPerson.Greet();

        Console.WriteLine($"GetRole() через посилання типу Person (new):
{studentAsPerson.GetRole()}");
        Console.WriteLine($"GetRole() через посилання типу Student (new):
{student.GetRole()}");
        Console.WriteLine($"GetRole() у Professor (успадкований):
{professor.GetRole()}");
    }
}
```

4. Опис класів

Person містить приватні поля `_name` та `_age`, публічні властивості `Name` і `Age` з валідацією, конструктор, віртуальний метод `Greet()` та звичайний метод `GetRole()`.

Student успадковує `Person`. Його конструктор викликає конструктор базового класу через `base(name, age)` і додатково встановлює `StudentId`. Клас перевизначає `Greet()` через `override`, має власний метод `Study()` та метод `new string GetRole()`, який приховує однойменний метод базового класу.

Professor успадковує `Person`, має властивість `Department`, викликає `base(name, age)`, перевизначає `Greet()` і має власний метод `Teach()`.

5. Результат виконання програми

Нижче наведено очікуваний вивід консолі.

```
=== Поліморфізм (override) ===
Привіт, мене звати Олег. Мій вік: 40.
Привіт, я студент Марія, студентський квиток ST-2025-07.
Доброго дня, я професор Іван Петренко, кафедра: Інформаційні технології.

=== Власні методи похідних класів ===
Марія вивчає об'єктно-орієнтоване програмування.
Іван Петренко читає лекцію на кафедрі: Інформаційні технології.

=== Різниця між override та new ===
Greet() через посилання типу Person (override):
Привіт, я студент Марія, студентський квиток ST-2025-07.
GetRole() через посилання типу Person (new): Особа
GetRole() через посилання типу Student (new): Студент
GetRole() у Professor (успадкований): Особа
```

Місце для скриншота консолі: вставте власний скриншот після запуску програми.

6. Висновок

У роботі реалізовано ієрархію `Person` → `Student`, `Professor`. Похідні класи викликають конструктор базового класу через `base(...)`, перевизначають віртуальний метод `Greet()` та мають власні унікальні методи.

Поліморфізм показано на масиві `Person[]`: для кожного елемента виконується версія `Greet()` фактичного типу об'єкта, хоча посилання має тип `Person`. Це можливо, бо метод позначений `virtual`, а в похідних класах — `override`, тому виклик визначається під час виконання.

Метод `GetRole()` показує іншу поведінку. Модифікатор `new` лише приховує метод базового класу: який варіант буде викликано, визначає тип посилання, а не тип об'єкта. Для `Person studentAsPerson = student` виклик `GetRole()` повертає «Особа», а через посилання типу `Student` — «Студент». Тому для поліморфної поведінки слід використовувати `virtual` та `override`, а `new` застосовувати лише тоді, коли потрібно навмисно приховати успадкований метод.